

CS633 Assignment Group Number 57

Ch V Sai Koushik (230312)
Challa Kethan (230317)
Chilamakuri Kundan Sai (230330)
Kamatam Akshay Reddy (230517)
Kishore Senthilkumar (230566)

15/02/2026

1 Code Description

1.1 Problem Overview

The program implements an MPI-based point-to-point data exchange and computation scheme using only blocking communication methods (`MPI_Send` and `MPI_Recv`). The objective is to exchange data (M doubles) between processes located at distances $D1$ and $D2$ to the right, perform distance-specific computations at the receiver ranks, and return the processed data to the sender ranks without any deadlock. This whole program is repeated for T iterations. The program's output will be a few statistics on the final state of the collected data after the last iteration.

1.2 Memory Allocation and Initialization

Buffer Strategy: The program utilizes a dedicated buffering strategy to ensure data consistency. Distinct arrays are allocated for:

- **Local State:** Holding the current iteration's data (M doubles).
- **Communication Channels:** Dedicated result buffers for the $D1$ and $D2$ topologies, alongside a shared buffer for initial data exchange.
- **Intermediate Results:** Temporary storage for computation results (Square/Log) before they are returned.

This explicit separation prevents data aliasing and ensures that the complex update logic (modulo/scaling) does not overwrite input data prematurely.

Rank Classification: To handle boundary conditions correctly, processes are classified into three logical roles based on their position relative to the total number of processes (P):

- **Full Sender:** If $rank + D2 < P$, the rank sends to both neighbors.
- **Partial Sender:** If $rank + D1 < P$ but $rank + D2 \geq P$, the rank sends only to $D1$.
- **Receiver Only:** If $rank + D1 \geq P$, the rank does not initiate any sends.

1.3 Core Idea

1.3.1 The Challenge

The primary constraint of the assignment is the exclusive use of **blocking communication** (`MPI_Send` and `MPI_Recv`). In a simple linear shift pattern where every process attempts to send to its D -distance

neighbor simultaneously, the system inevitably enters a **circular wait deadlock**. Every process enters a blocking send state, waiting for a destination process that is itself blocked trying to send to another rank.

1.3.2 Two-Phase Execution

The implementation is based on the idea of **separating communication for $D1$ and $D2$ distances** and handling each distance independently within each iteration. First, we perform all the communications for $D1$ and later for $D2$ in a similar procedure. This approach avoids deadlocks, message interference, and allows each cycle of communication passage, computation of values, and returning them to be completed fully.

For each distance ($D1$ or $D2$), the execution follows the same structured sequence:

1. Forward communication from sender to receiver
2. Computation at the receiver
3. Backward communication from receiver to sender
4. Update of sender buffers for the next iteration

1.3.3 Group-Based Parity Scheduling

To overcome the challenge of deadlocks in each phase, we opted for a **parity-based communication pattern** based on grouping ranks using floor division:

$$g = \left\lfloor \frac{\text{rank}}{D} \right\rfloor$$

where D is either $D1$ or $D2$.

This grouping ensures that any rank in group g will always target a receiver in the adjacent group $g + 1$. For each distance, communication is divided into two steps:

- **Step 1:** Ranks with even group index ($g \bmod 2 = 0$) perform `MPI_Send` to their `Rank + D` who now belongs to right neighbor group, while odd groups perform `MPI_Recv`.
- **Step 2:** The roles are reversed: odd groups send and even groups receive.

1.3.4 Justification for Optimality

The main factors that make this scheduling approach ideal for the constraints are:

1. **Guaranteed Deadlock Freedom:** The circular wait condition is mathematically impossible since it is strictly enforced that an `MPI_Send` is never posted unless the corresponding receiver is already in a `Recv` state (or entering it right away in the current phase).
2. **Maximum Concurrency:** All ranks cannot send simultaneously during a blocking linear shift. The theoretical throughput limit for this topology is reached by using a bipartite approach using parity, which permits approximately 50% of the process pairs to communicate in parallel.

1.4 Receiver Side Computation and Backward Communication

On receiving the data from sender, the receiver performs the required computation:

- For distance $D1$, each element is squared.
- For distance $D2$, the logarithm of each element is computed

The computation is performed locally and stored in a reply buffer, which is then sent back to the original sender using the same parity based ordering. Please note that by computing after receiving of all the ranks occur, the implementation ensures that all ranks remain active during the computation stage at once.

1.5 Sender Side Update

Only ranks that are valid senders for a given distance will receive a reply buffer. These ranks update their send buffers for the next iteration according to the specified rules:

- For D_1 , values are typecast to unsigned integers and then reduced using modulo 100,000.
- For D_2 , values are scaled by a factor of 100,000.

Ranks that are not valid senders skip the update.

1.6 Final Aggregation

After completing T iterations, each valid sender computes the maximum value across its final received arrays for D_1 and D_2 . These local maxima are then communicated to rank 0. The root process receives these values exclusively from valid senders, calculates the global maximum for both D_1 and D_2 , and outputs the results in the format: `<maximumD1> <maximumD2> <time>`.

The `time` parameter represents the total execution duration, measured from the start of the first iteration.

***Note:** Data sent to rank 0 after T iterations is exempt from the “Sender Side Update” logic.*

2 Code Compilation and Execution Instructions

The experimental evaluation was conducted on the **Param Rudra** supercomputing facility (CDAC Delhi). This section details the build environment, compilation and the job submission pathway employed to ensure reproducible results.

2.1 Environment Configuration

The cluster utilizes the OneAPI toolkit for MPI implementation. Before compilation, the user environment must be configured to load the necessary compiler and library paths.

```
module load compiler/oneapi-2024/mpi
```

2.2 Compilation Procedure

The source code `src.c` is compiled using the MPI wrapper (`mpicc`). We utilize specific compiler flags to optimize for the target architecture and link necessary mathematical libraries.

- **-o src:** Specifies the output binary name.
- **-lm:** Explicitly links the math library. This is critical as the receiver ranks compute logarithms (`log()`) during the data transformation phase.
- **-O2 (optional):** While not strictly mandatory, using optimization level 3 is a good practice in HPC, which benefits the `square` and `log` computations.

Command:

```
mpicc -O2 -o src src.c -lm
```

2.3 Execution and Job Submission

The program accepts five command line arguments to allow for flexible testing without recompilation:

```
./src <M> <D1> <D2> <T> <seed>
```

Arguments:

1. **M (Data Size):** The number of double values per rank.(262,144 (2^{18}) and 1,048,576 (2^{20})).
2. **D1, D2 (Distances):** Neighbor offsets, fixed at 2 and 4 respectively.
3. **T (Iterations):** The number of compute-communicate cycles, fixed at 10.
4. **Seed:** The random seed generator, fixed at 1000 to ensure data consistency across runs.

2.4 SLURM Batch Scripting

To avoid interference from other users, jobs were submitted via the SLURM scheduler. We were bounded to the constraint of **16 processes per node**. This mapping is critical for analyzing the results as it defines when communication crosses the boundaries of the physical nodes.

For the maximum configuration of $P = 32$, the job spans exactly two compute nodes. Below is the content of the `job_p32.sh` script used for the final benchmarks:

```
#!/bin/bash
#SBATCH --job-name=assign1_p32
#SBATCH --nodes=2
#SBATCH --ntasks=32
#SBATCH --ntasks-per-node=16
#SBATCH --time=00:10:00
#SBATCH --output=output1_p32.txt

module load compiler/oneapi-2024/mpi

# for Small M
for i in {1..5}
do
    mpirun -np 32 ./src 262144 2 4 10 1000
done

# for Large M
for i in {1..5}
do
    mpirun -np 32 ./src 1048576 2 4 10 1000
done
```

The jobs were submitted using the `sbatch job_p32.sh` command. Job output file (`job_%j.out`) will contain the required program prints; the final single-line output from rank 0 follows the required format:

```
<maximumD1> <maximumD2> <time>
```

3 Results

This section presents a detailed quantitative and qualitative analysis of the application’s performance. For each configuration the experiment was repeated five times and recorded the program reported elapsed time. The independent variables were number of processes (P) and data-size (M) and the parameters D1, D2, T, seed were fixed. The experiments investigate the scalability of the solution across three process counts ($P \in \{8, 16, 32\}$) and two sizes ($M \in \{262144, 1048576\}$).

The lines below are sample final-output lines (one per run) in the format `<maxD1> <maxD2> <time>`.

```
9987204096.000000 14.163601 0.993701
9987204096.000000 14.163601 1.017912
...
```

Note that all timing data reported are the aggregate execution time for the complete program (Initialization + T Iterations + Reduction). The final aggregated timing file follows the CSV format:

```
P, Size, T1, T2, T3, T4, T5
8, small, 0.089882, 0.090529, 0.089687, 0.089265, 0.089605
8, large, 0.340586, 0.341896, 0.340645, 0.340876, 0.341363
...
```

3.1 Correctness

Before analyzing performance, the logical correctness of the implementation can be verified. Across all 30 experimental runs (3 configurations $\times$ 2 data sizes $\times$ 5 repetitions), the final reduced output at Rank 0 remained constant.

Table 1: Output Verification Logs

Metric	Value
Max Data Received (D1)	9987204096.000000
Max Data Received (D2)	14.163601

This consistency confirms that:

- The random number generation is correctly seeded and distributed based on rank.
- The "Invalid Sender" logic ($rank + D > P - 1$) correctly prevents boundary ranks from sending data while allowing them to receive, ensuring no data corruption.
- The iterative updates (modulo and multiplication) are being applied in the correct order.

3.2 Quantitative Performance Data

Table 2 summarizes the execution time statistics. The "Average Time" and "Standard Deviation" were calculated from 5 independent runs per configuration to ensure statistical significance.

Table 2: Performance Statistics (Time in Seconds)

Processes (P)	M = 262,144		M = 1,048,576	
	Avg Time (s)	Std Dev (σ)	Avg Time (s)	Std Dev (σ)
8	0.089794	± 0.000468	0.341073	± 0.000552
16	0.396306	± 0.087499	0.706614	± 0.013015
32	0.738284	± 0.027414	1.029462	± 0.042560

3.3 Visual Analysis

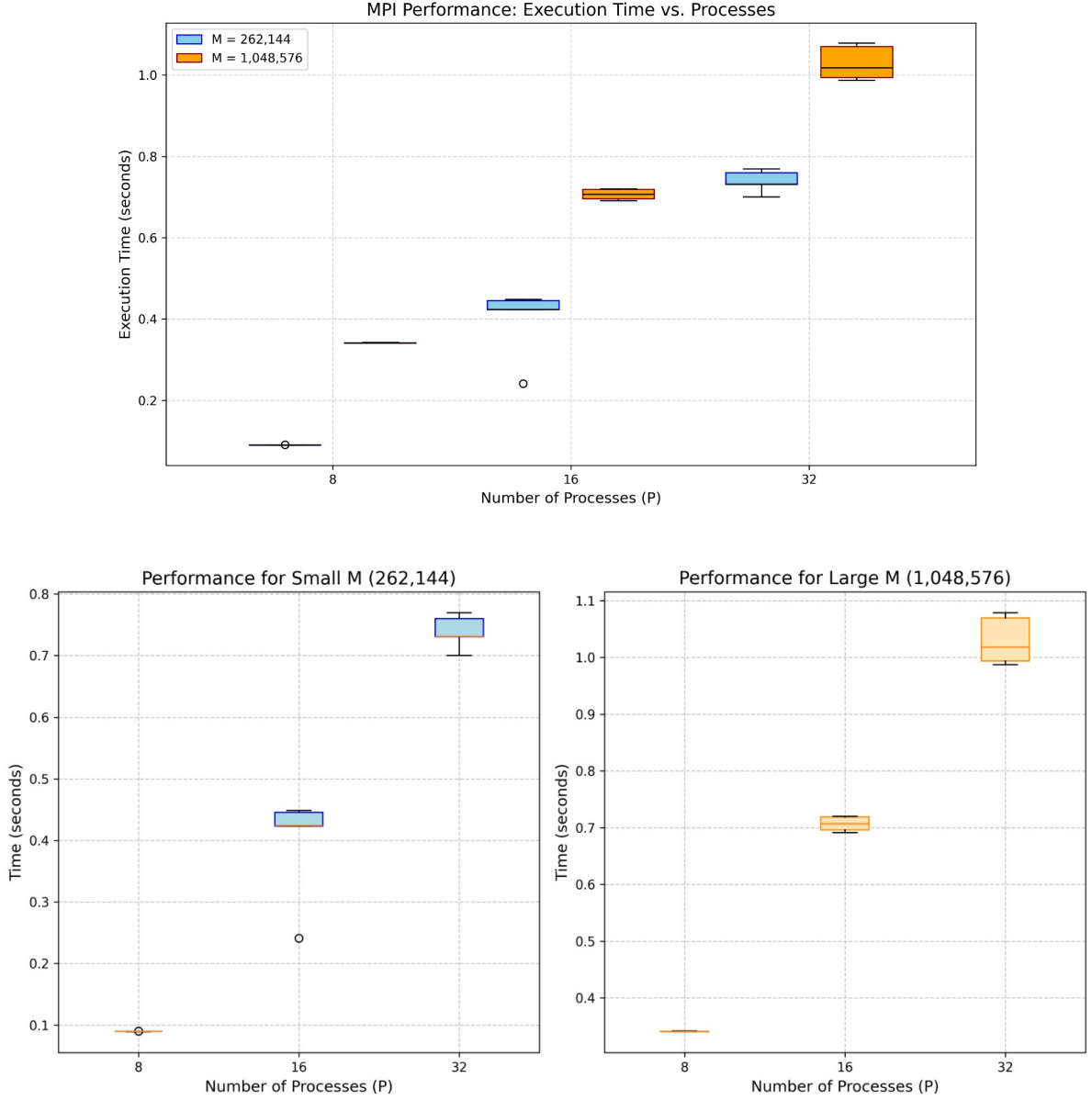


Figure 1: Execution time distribution (in seconds) across 5 independent runs. The top plot overlays both workloads, while the bottom panels separate them for clarity.

The above box plots visualizes the performance trends. The x and y axes representing the process counts ($P \in \{8, 16, 32\}$) and the total execution time (seconds). The workloads are color coded as **blue** for the smaller dataset ($M = 262,144$) and **orange** for the larger dataset ($M = 1,048,576$). Each boxplot displays the median (central line), the Interquartile Range (box limits), and the full data range (whiskers). Outliers, representing runs affected by transient system noise, are marked as distinct circles (visible at $P = 8, 16$).

3.4 Analysis of Results and Code Efficiency

In the given problem statement, since every rank manages its own array of M , increasing the number of processes (P) increases the total data volume and computation across the system, while the workload for each individual process remains constant. Ideally, if communication were instantaneous and bandwidth

infinite, the time should remain constant. However, our results show a clear increase in runtime. We can attribute this to multiple distinct algorithmic and hardware level bottlenecks.

3.4.1 Intra Node Saturation

One of the notable observations is the jump in execution time when moving from $P = 8$ to $P = 16$ for the smaller dataset ($M = 262,144$).

- **Observation:** Time increased from $0.09s$ to $0.39s$ - a nearly $4\times$ slowdown.
- **Analysis:** Since we are running 16 processes per node, at $P = 8$, only half the cores on the node are active, leaving ample memory bandwidth available per core. However, at $P = 16$, the node is fully saturated. All 16 cores are simultaneously attempting to read and write the large array to the shared cache and main RAM.
- **Conclusion:** The sharp performance drop confirms that the node has reached its **memory bandwidth limit**. With all 16 cores accessing RAM simultaneously, the shared memory controller becomes a bottleneck, starving the CPUs of data and causing significant execution stalls.

3.4.2 Code Inefficiency due to Algorithmic Constraints

Blocking Synchronization Overhead: The strict requirement to use blocking calls (`MPI_Send` / `MPI_Recv`) enforces a "stop-and-wait" workflow. Because a sender cannot proceed until its receiver is ready, the system cannot perform useful computations while data is moving through the network. This prevents the program from masking communication delays, which becomes significant.

3.4.3 Interconnect Latency (Inter-Node Communication)

When scaling from $P = 16$ to $P = 32$, the physical topology changes from a single node to two nodes.

- **Observation:** For $M = 1,048,576$, time increased from $0.70s$ to $1.03s$.
- **Analysis:** At $P = 32$, ranks near the boundary (e.g., Rank 15 and Rank 16) must exchange data across the **network interconnect** rather than through shared memory. Inter node latency is lot higher than intra node latency.
- **Impact:** This introduces serialization delays. Since MPI collective performance is often limited by the slowest link, the latency of the cross-node communication slows down the entire iterative cycle for all ranks, resulting in the observed increase in total runtime.

3.4.4 Data Size Scaling

Comparing the two dataset sizes provides insight into the algorithmic complexity.

- The ratio of data sizes is $1048576/262144 = 4$.
- At $P = 8$, the time ratio is $0.34s/0.09s \approx 3.9$.

This near-linear scaling ($3.9 \approx 4$) confirms that for low process counts (where contention is low), the algorithm is compute-bound and scales predictably. However, at $P = 32$, the ratio drops ($1.03s/0.73s \approx 1.45$), indicating that at scale, some fixed overheads begin to dominate over raw data processing time.

4 Conclusions

In this assignment, we successfully designed, implemented, and benchmarked a distributed MPI program to perform iterative data exchange and computation on the **Param Rudra** supercomputing cluster. The result outputs and time analysis were shown and done across all 30 execution runs for various configurations.

Few conclusions that can be made are:

- **Impact of Increasing Processes (P) with Fixed Data (M):** The execution time increased as we added more processes (as every new process adds more total work to the system). As the number of processes grew from 8 to 32, the overhead of coordinating communication first within a node and then inter-node communication time began to outweigh the speed of computation.
- **Impact of Increasing Data Size (M) with Fixed Processes (P):** The execution time increased when the data size was quadrupled (as larger arrays require proportionally more time to compute and take longer to transmit between ranks).

4.1 Contributions

Table 3: Team Member Contributions

Roll No.	Name	Contribution %
230312	Ch V Sai Koushik	20%
230317	Challa Kethan	20%
230330	Chilamakuri Kundan Sai	20%
230517	Kamatam Akshay Reddy	20%
230566	Kishore Senthilkumar	20%

(Note: All members contributed equally to the analysis and final compilation of this Assignment.)